LAPORAN PRAKTIKUM & TUGAS PRAKTIKUM

PEMROGRAMAN

BERORIENTASI OBJEK

Modul 1 - 5

Disusun Oleh :

Razif Al Farisi

2511532028



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

Langkah Kerja Praktikum

A. Membuat Class rekening (tempat data-data akun)

1. Membuat class dengan nama rekening.java
2. Menambahkan atribut namaPemilik, nomorRekening dengan tipe String dan saldo dengan tipe double.

```
public class rekening {

    String nomorRekening;
    String namaPemilik;
    double saldo;
```

3. Membuat sebuah constructor (constructor harus sama dengan nama class. Misalnya nama class anton, berarti constructornya public anton()). Constructor diberi sebuah parameter untuk menerima dan menyimpan data akun pemilik.

```
public rekening(String nomor, String nama, double saldoAwal) {
    nomorRekening = nomor;
    namaPemilik = nama;
    saldo = saldoAwal;
    System.out.println("Rekening atas nama : " + namaPemilik + " Berhasil dibuat dengan saldo Rp" + saldo);
}
```

4. Membuat method getNama dan getNoRek untuk menampilkan nama dan norekening dari akun pemilik yang aktif saat ini.

```
public String getNoRek() {return nomorRekening;}
public String getNama() {return namaPemilik;}
```

5. Membuat method setorTunai() dengan parameter double. Fungsinya disini untuk menyetorkan tunai ke akun bank sang pemilik. Method ini menggunakan if untuk memeriksa apakah nominal yang di inputkan sesuai aturan atau tidak (nominal > 0) dan penambahan saldo menggunakan *OperatorAssignment*.

```
public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
    System.out.println("Setor tunai Rp" + nominal + "berhasil. saldo saat ini Rp" + saldo);
    } else {
        System.out.println("Gagal: nominal setor harus lebih dari 0!");
    }
}
```

6. Membuat method cekInformasi() dengan memanggil atribut yang sudah dibuat sebelumnya. Tujuannya melihat detail akun bank seperti nama, norekening, dan saldo dari akun pemilik.

```
public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening    : " + nomorRekening);
    System.out.println("Nama Pemilik    : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("----------------------");
}
```

7. Lalu membuat method tarikTunai() dengan parameter double. Method ini menggunakan logika if dimana kalau saldo yang dimiliki kurang dari 10000 itu tidak bisa melakukan penarikan. Tetapi jika sebaliknya, pemilik dapat melakukan penarikan sesuai aturan.

```
public void tarikTunai(double nominal) {
    if (saldo <= 10000) {
        System.out.println("Saldo Anda tidak mencukupi untuk menarik tunai anda!");
    } else {
        if (nominal >= saldo) {
            System.out.println("Saldo Anda tidak Cukup!");
        } else {
            saldo -= nominal;
            System.out.println("Saldo Anda saat ini : Rp" + saldo);
        }
    }
}
```

B. Membuat class main.java (class utama)

1. Membuat sebuah objek dengan nama akunAktif dan sebuah Scanner untuk menginputkan sesuatu lansung dari keyboard(untuk scanner jangan lupa membuat import java.util.Scanner; di luar class / di atas class). Atribut boolean untuk menjalankan program tersebut dan menampilkan menu menu yang ada

```
package Modul_1;

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        rekening akunAktif = null;
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\n Menu Utama :");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik TUnai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("0. Keluar");
            System.out.print("Pilih Menu : ");

            int pilihan = input.nextInt();
            input.nextLine();
```

2. Int pilihan untuk memilih menu yang ingin diakses sesuai nomor yang di tampilkan.
3. Pilihan 1 digunakan untuk menambahkan / membuka rekening baru, nama, norekening, dan saldo diinputkan lansung dan disimpan melalui pembuatan objek (newrekening (parameter)).

```
        switch (pilihan) {
            case 1 :
                System.out.print("Masukkan No Rekening :");
                String no = input.nextLine();
                System.out.print("Masukkan Nama Pemilik : ");
                String nama = input.nextLine();
                System.out.print("Masukkan Saldo Awal : ");
                double saldo = input.nextDouble();

                akunAktif = new rekening(no, nama, saldo);
                break;
```

4. Untuk pilihan ke-2, digunakan untuk melakukan penyetoran tunai ke rekening yang sudah dibuat sebelumnya. Kalau rekening belum dibuat, penyetoran tidak dapat dilakukan (akunAktif == null). Nominal penyetoran menggunakan tipe data double.

```
case 2:
    if (akunAktif == null) {
        System.out.println("Error : Mohon maaf, Anda belum memiliki nomor rekening!");
    } else {
        System.out.print("Masukkan nominal setor : ");
        double setor = input.nextDouble();
        akunAktif.setorTunai(setor);
    }
    break;
```

5. Pada pilihan ke-3, digunakan untuk melakukan penarikan tunai. Penarikan tersebut harus sesuai dengan aturan yang sudah di buat di class rekening.java sebelumnya (saldo harus lebih dari 10000). Lalu method tarikTunai dari objek yang sudah dibuat dipanggil untuk menyimpan perubahan.

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error : Mohon maaf, Anda belum memiliki nomor rekening!");
    } else {
        System.out.print("Masukkan saldo yang di tarik : Rp");
        double nominal = input.nextDouble();
        akunAktif.tarikTunai(nominal);
    }
    break;
```

6. Pilihan ke-4, digunakan untuk melihat detail-detail dari akun yang sudah di buat, mulai dari nama pemilik, norekening, dan saldo didalam akun tersebut.

```
case 4:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        akunAktif.cekInformasi();
    }
    break;
```

7. Lalu pilihan 0 ini untuk keluar dari sistem / program. Selain dari itu inputan akan menampilkan pesan bahwa pilihan yang dipilih tidak valid. Untuk pilihan ke-5(ganti akun) dan ke-3(tarikTunai) akan di tampilkan di bagian penugasan praktikum.

```
        case 0:
            isRunning =false;
            System.out.println("Sistem ditutup. Terimakasih");
            break;

        default:
            System.out.println("Pilihan tidak valid");

    }
```

Langkah Kerja Praktikum 2

A. Membuat Class transaksi (tempat detail transaksi)

1. Menambahkan atribut idTransaksi dan jenis bertipe String. Lalu atribut nominal bertipe double.
2. Lalu membuat sebuah konstruktor untuk menyimpan data transaksi tersebut

```
public class transaksi {
    String idTransaksi;
    String jenis;
    double nominal;

    // constructor
    public transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }
```

3. Menggunakan method getId & getNominal untuk menampilkan id transaksi dan nominal dari transaksi pada akun yang aktif saat ini.

```
public String getId() {return this.idTransaksi;}
public double getNominal() {return this.nominal;}
```

4. Lalu membuat method cetakDetail() untuk menampilkan id transaksinya, jenis transaksinya, dan nominalnya berapa.

```
public void cetakDetail() {
    System.out.println("ID : " + idTransaksi + " | Jenis : "
+ jenis + " | Nominal : Rp" + nominal);
}
```

B. Membuat Relasi pada Rekening.java (menghubungkan class transaksi dan rekening)

1. Mendeklarasikan variable riwayatTransaksi dengan struktur data Arraylist untuk menyimpan semua data transaksi ke akun yang aktif saat ini.

```java
public class rekening {
    String nomorRekening;
    String namaPemilik;
    double totalSetor;
    double totalTarik;
    double saldo;

    ArrayList<transaksi> riwayatTransaksi;
```

2. Lalu menambahkan kode pada konstruktor untuk membuat sebuah objek dari riwayatTransaksi tersebut. Saat konstruktor di panggil, riwayat transaksi akan membuat sebuah objek.

```java
 public rekening(String nomor, String nama, double saldoAwal) {
     nomorRekening = nomor;
     namaPemilik = nama;
     saldo = saldoAwal;

     this.riwayatTransaksi = new ArrayList<>();

     System.out.println("Rekening atas nama : " + namaPemilik +
 }
```

3. Menambahkan idTrx pada method setorTunai dan tarikTunai. Saat melakukan tarik atau setor tunai, method tersebut akan membuat idtransaksi dalan variable idTrx. Tiap id berbeda tipe (TRX-S- untuk setor tunai & TRX-T- untuk tarik tunai). Lalu data idTrx tersebut di tambahkan ke dalam objek riwayatTransaksi serta jenis transaksi dan nominalnya.

```java
public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        String idTrx = "TRX-S-" + System.currentTimeMillis();
        transaksi trxBaru = new transaksi(idTrx, "Kredit", nomin
        riwayatTransaksi.add(trxBaru);
```

```java
public void tarikTunai(double nominal) {
    if (saldo <= 10000) {
        System.out.println("Saldo Anda tidak mencukupi untuk menarik tunai anda!");
    } else {
        if (nominal >= saldo) {
            System.out.println("Saldo Anda tidak Cukup!");
        } else {
            saldo -= nominal;
            String idTrx = "TRX-T-" + System.currentTimeMillis();
            transaksi trxBaru = new transaksi(idTrx, "Dedit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Saldo Anda saat ini : Rp" + saldo);
        }
```

4. lalu method cetakMutasi(), ini merupakan tugas pada modul 2.

```java
    public void cetakMutasi() {
        System.out.println("-------Riwayat Transaksi-------");
        int start = 0;
        if (riwayatTransaksi.size() > 3)
            start = riwayatTransaksi.size() - 3;

        for (int i = start; i < riwayatTransaksi.size(); i++)
            riwayatTransaksi.get(i).cetakDetail();
    }

}
```

Langkah Kerja Praktikum 3

A. Membuat attribut di dalam class transaksi.java terkunci (enkapsulasi)

1. Menambahkan kode “private” untuk setiap attribut di transaksi

```
public class transaksi {
    private String idTransaksi;
    private String jenis;
    private double nominal;
```

2. Untuk bisa di akses nilai dari attribut tersebut atau membaca nilai attributnya. Ditambahkan fungsi getter untuk setiap transaksi.

```
    public String getId() {return this.idTransaksi;}
    public String getJenis() {return this.jenis;}
    public double getNominal() {return this.nominal;}

    public void cetakDetail() {
        System.out.println("ID : " + idTransaksi + " | Jenis : "
    }
```

Jadi attribut tersebut tidak ditambahkan method setter, karena attribut sudah bersifat tidak boleh di ubah.

B. Mengunci attribut rekening dan menambahkan sistem PIN

1. Menambahkan “private’ untuk mengunci attribut-attribut tersebut. Lalu menambahkan method getter untuk membaca nilai attribut tersebut

```
public class rekening {
    private String nomorRekening;
    private String namaPemilik;
    private double totalSetor;
    private double totalTarik;
    private double saldo;
    private String pinbaru;
    private String pinLama;

    ArrayList<transaksi> riwayatTransaksi;
```

```
public String getNoRek() {return nomorRekening;}
public String getNama() {return namaPemilik;}
public double getNominal() {return saldo;}
```

2. Lalu tambahkan attribut PINbaru untuk menyimpan PIN yang akan digunakan pengguna untuk menarik tunai
3. Buat sebuah method bertipe boolean untuk melakukan pengecekan terhadap PIN yang dimasukkan agar dapat mengakses tarikTunai dan cekMutasi(ini termasuk ke bagian tugas).

```
public boolean otentikasi(String inputPin) {
    return this.pinbaru.equals(inputPin);
}
```

4. Untuk hasil dan penambahan kode pada menu tarikTunai, cetakMutasi , dan saat pembuatan akun akan dipaparkan di bagian tugas/latihan.

Langkah Kerja Praktikum 4

A. Memodifikasi atribut pada class rekening.java

1. Menambahkan hak akses protected ke atribut saldo dan riwayaTransaksi. Jadi yang hanya bisa mengakses class dalam package yang sama atau subclass(turunan)dari class ini.

```
protected double saldo;
protected ArrayList<transaksi> riwayatTransaksi;
```

B. Membuat class RekeningTabungan.java (turunan dari rekening.java)

1. Pada public class tersebut di tambahkan kode relasi “extends rekening”. Maksudnya Rekening tersebut mewarisi atribut dan method dari si induk (rekening.java), sesuai dengan hak akses yang dimiliki atribut / method.
2. Lalu class ini terdapat atribut sukuBunga dengan tipe double dan hak akses private.
3. Untuk parameter konstruktor disamakan dan ditambah dengan parameter sukuBunga. Saat dipanggil data yang dinputkan dan atribut tambahan khusus untuk rekeningTabungan(sukuBunga) disimpan.

```
package Modul_4;

public class RekeningTabungan extends rekening {

    private double sukuBunga;

    public RekeningTabungan(String nomor, String nama, double saldo

        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }
```

4. Lalu buat method tambahBungaAkhriBulan dengan return type void. Dalam method terdapat local variable yang berfungsi menyimpan nominalBunga yang didapat dari saldo dikalikan dengan berapa persen sukubunga(saldo * (sukuBunga/100)). Lalu saldo dari rekening user

akan ditambah dengan nominalBunga tadi. Idtransaksi untuk rekeningTabungan ini adalah “TRX-B-“ dan membuat objek transaksi yang menyimpan idtransaksi, bunga, dan nominalBunga tersebut serta menambahkan ke kumpulan data (riwayatTransaksi).

```
public boolean otentikasi(String inputPin) {
    return this.pinbaru.equals(inputPin);
}
```

5. Untuk hasil dan penambahan kode pada menu tarikTunai, cetakMutasi , dan saat pembuatan akun akan dipaparkan di bagian tugas/latihan.

```
public void tambahBungaAkhirBulan() {
    double nominalBunga = saldo * (sukuBunga / 100);
    saldo += nominalBunga;

    String idTrx = "TRX-B-" + System.currentTimeMillis();
    riwayatTransaksi.add(new transaksi(idTrx, "Bunga", nominalBunga));

    System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBur
}
```

C. Membuat class RekeningGiro.java(turunan rekening.java)

1. Membuat class dan menambahkan kode “extends rekening” di public class RekeningGiro. Lalu membuat atribut batasOverdraft dengan tipe double dan hak akses private.
2. Membuat konstruktor dengan parameter seperti induk yang ditambah dengan parameter batasOverdraft(double). Lalu konstruktor induk dibuat menggunakan perintah super() dan data batasOverdraft disimpan dalam subclass.
3. Setelah itu method getter untuk mengambil atau membaca nilai dari atribut batasOverdraft.

```
package Modul_4;

public class RekeningGiro extends rekening{
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    public double getBatasOverdraft() {
        return batasOverdraft;
    }

}
```

Langkah Kerja Praktikum 5

A. Memodifikasi method tarikTunai() pada class rekening.java

1. Mengubah tipe return dari void menjadi boolean pada method tersebut.
2. Lalu tiap logika yang menghasilkan nilai false atau yang membuat sistem gagal itu diberikan return false;. dan kalau berhasil return true.

```
public boolean tarikTunai(double nominal) {
    if (saldo <= 10000) {
        System.out.println("Saldo Anda tidak mencukupi untuk m
        return false;
    } else if(nominal <= 0){
        System.out.println("Gagal: Nominal harus lebih dari 0"
        return false;
    } else if (nominal > saldo) {
        System.out.println("Saldo Anda tidak Cukup!");
        return false;
    } else {
        saldo -= nominal;
        String idTrx = "TRX-T-" + System.currentTimeMillis();
        transaksi trxBaru = new transaksi(idTrx, "Debit", nomi
        riwayatTransaksi.add(trxBaru);

        System.out.println("Saldo Anda saat ini : Rp" + saldo)
        return true;
    }
```

3. Setelah itu lakukan overriding pada class rekeningGiro.java
4. Tambahkan method tarik tunai sesuai dengan method class induknya. Namun tambahkan @Overriding sebelum method tersebut.

```
@Override
public boolean tarikTunai(double nominal) {
    if(saldo <= 10000) {
        System.out.println("Gagal: saldo harus lebih dari 10000!")
        return false;
    } else if(nominal <= 0) {
        System.out.println("Gagal: nominal harus lebih dari 0!");
        return false;
    } else if(nominal > (saldo + batasOverdraft)) {
        System.out.println("Transaksi Gagal: Melebihi batas overdr
    + (saldo + batasOverdraft));
        return false;
    } else {
        saldo -= nominal;

        String idTrx = "TRX-G-" + System.currentTimeMillis();
        transaksi trxBaru = new transaksi(idTrx, "Debit Giro", nor
        riwayatTransaksi.add(trxBaru);

        System.out.println("Tarik tunai Giro Rp" + nominal + " ber
        return true;
    }
```

5. Di class ini, logika (nominal > saldo) diganti menjadi (nominal > (saldo + batasOverdraft)). Jadi pengguna diizinkan menarik lebih dari saldo yang mengakibatkan saldo minus.
6. Untuk uji cobanya seperti berikut :

```
Pilih Menu : 1
Masukkan No Rekening :budi01
Masukkan Nama Pemilik : budi
Masukkan Saldo Awal : 100000
Buat PIN : 121212
Pilih tabungan yang ingin di tambahkan: 2
Masukkan BatasOverdraft: 400000
Rekening atas nama : budi Berhasil dibuat dengan saldo Rp100000.0
```

```
Pilih Menu : 3
Masukkan PIN :
121212
Masukkan saldo yang di tarik : Rp500000
Tarik tunai Giro Rp500000.0 berhasil. Saldo saat ini Rp-400000.0
Masukkan saldo yang di tarik : Rp100000
Transaksi Gagal: Melebihi batas overdraft! Maksimal penarikan: Rp0.0
```

Tugas dan Latihan Praktikum

A. Implementasi dari tarikTunai

```
public void tarikTunai(double nominal) {
    if (saldo <= 10000) {
        System.out.println("Saldo Anda tidak mencukupi untuk menarik tunai anda!");
    } else {
        if (nominal >= saldo) {
            System.out.println("Saldo Anda tidak Cukup!");
        } else {
            saldo -= nominal;
            System.out.println("Saldo Anda saat ini : Rp" + saldo);
        }
    }
```

Implementasi dari method **tarikTunai** ini menggunakan parameter double sesuai dengan tipe data dari variable **saldo. Logika if** digunakan untuk memeriksa apakah saldo yang dimiliki mencukupi syarat melakukan penarikan (saldo <=10000). Jika bernilai benar method akan mengembalikan berupa pesan kalau saldo yang dimiliki tidak mencukupi. Jika bernilai salah, penarikan bisa dilanjutkan. Untuk percobaannya seperti berikut :

```
Pilih Menu : 1
Masukkan No Rekening :19200
Masukkan Nama Pemilik : tony
Masukkan Saldo Awal : 15000
Rekening atas nama : tony Berhasil dibuat dengan saldo Rp15000.0
```

Saat ini pemilik bernama tony memiliki saldo sebesar Rp15.000 . Tony akan melakukan penarikan sebesar Rp10.000 . Terlihat pada gambar berikut bahwa saldo yang dimiliki tersisa Rp5.000 .

```
0. Keluar
Pilih Menu : 3
Masukkan saldo yang di tarik : Rp10000
Saldo Anda saat ini : Rp5000.0
```

```
--- INFO REKENING ---
No. Rekening    : 19200
Nama Pemilik    : tony
Saldo Akhir     : Rp5000.0
---------------------
```

Saat melakukan penarikan lagi dengan nominal Rp1 . Sistem menampilkan pesan bahwa saldo Tony tidak mencukupi untuk melakukan tarikTunai.

```
Masukkan saldo yang di tarik : Rp1
Saldo Anda tidak mencukupi untuk menarik tunai anda!
```

Jadi, sebesar apapun penarikan dilakukan atau sekecil apapun kalau saldo yang dimiliki kurang dari Rp10.001, maka sistem tidak dapat melanjutkan penarikan.

B. Tantangan Multi-Akun

Pada penerapan ini, beberapa atribut atau kode program berubah seperti ada nya objek ArrayList untuk menyimpan akun-akun yang dibuat pemilik. Pertama dimulai dari class **rekening.java.** pada class ini hanya sedikit penambahan berupa method gantiAkun dan import ArrayList agar parameter Arraylist tetap berjalan.

```
import java.util.ArrayList;

public class rekening {
```

```
public rekening gantiAkun(ArrayList<rekening> daftar, int index) {
    if (index >= 0 && index < daftar.size()) {
        System.out.println("Berhasil pindah ke akun: " + daftar.get(index).getNama());
        return daftar.get(index);
    } else {
        System.out.println("Gagal: akun tidak ditemukan!");
        return null;
    }
}
```

Pada method ini dibutuhkan sebuah index untuk berganti akun. Parameter ArrayList tersebut akan diambil dari objek penyimpanan akun-akun yang terletak pada class main.java. Kalau index yang diinputkan itu kecil dari 0 dan besar dari kapasitas / rentang dari penyimpanan ArrayList tersebut. Maka pergantian akun gagal dilakukan karena tidak menemukan pilihan yang valid. Selanjutnya pada class main.java.

```java
import java.util.Scanner;
import java.util.ArrayList;

public class Main {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<rekening> akunSaya = new ArrayList<>();
        rekening akunAktif = null;
        boolean isRunning = true;
```

Pada main.java ini objek yang digunakan 2, yaitu tipe ArrayList dengan nama akunSaya untuk menyimpan akun-akun yang sudah dibuat dan akunAktif untuk menunjukkan akun yang sedang dipakai saat ini. Untuk mengganti akun, opsinya diletakkan pada nomor ke-5.

```java
case 5:
    System.out.print("Akun saat ini : " + akunAktif.getNama() + " - " + akunAktif.getNoRek()+ "\n");
    System.out.println("---------Daftar Akun---------");
    for (int i = 0; i < akunSaya.size(); i++) {
        System.out.print((i+1) + ". Nama Pemilik Akun   : " + akunSaya.get(i).namaPemilik + " - Nomor Rekening : " + akunSaya.get(i).getNoRek());
        System.out.println();
    }
    System.out.print("Pilih Akun :");
    int pilihAkun = input.nextInt() - 1;
    noAkun = pilihAkun;
    akunAktif = akunAktif.gantiAkun(akunSaya, pilihAkun);
    break;
```

Disini method getNama dan getNoRek digunakan agar hasilnya tidak berupa hashcode. Perulangan for untuk mengakses akun-akun di dalam ArrayList yang sudah di buat. Agar penomoran lebih bagus, int i tersebut di tambah 1 (+1) saat menampilkan hasil, begitu juga dengan pemilihannya yang di kurangi 1 (-1) agar opsi yang dipilih sesuai nomor dan index yang di ambil. Lalu objek **akunAktif** tadi berganti dengan akun yang sudah dipilih. Berikut contoh penerapannya :

```
Pilih Menu : 1
Masukkan No Rekening :19211
Masukkan Nama Pemilik : tony
Masukkan Saldo Awal : 15000
Rekening atas nama : tony Berhasil dibuat dengan saldo Rp15000.0
```

```
Masukkan No Rekening :10251
Masukkan Nama Pemilik : Jaki
Masukkan Saldo Awal : 21000
Rekening atas nama : Jaki Berhasil dibuat dengan saldo Rp21000.0
```

```
 Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik TUnai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih Menu : 5
Akun saat ini : tony - 19211
---------Daftar Akun---------
1. Nama Pemilik Akun     : tony - Nomor Rekening : 19211
2. Nama Pemilik Akun     : Jaki - Nomor Rekening : 10251
Pilih Akun :2
Berhasil pindah ke akun: Jaki
```

Disini ada 2 akun yang sudah di buat yaitu dengan nama tony dan Jaki. Saat memilih pilihan 5, daftar akun di tampilkan. Urutan akunnya sesuai dengan urutan akun yang dibuat pertama / duluan. Akun yang pertama dibuat otomatis menjadi akun yang aktif saat itu walaupun banyak akun di tambahkan, akun yang aktif tidak akan berpindah sebelumnya melakukan pergantian pada pilihan 5. Kemudian akun yang dipilih adalah nomor 2 (nama : Jaki – NoRek : 10251), akun tersebut sudah berganti ke milik si Jaki.

```
 Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik TUnai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih Menu : 5
Akun saat ini : Jaki - 10251
---------Daftar Akun---------
1. Nama Pemilik Akun     : tony - Nomor Rekening : 19211
2. Nama Pemilik Akun     : Jaki - Nomor Rekening : 10251
```

Begitu untuk konsep method gantiAkun yang saya buat. Semoga apa yang saya paparkan dapat menjadi penilaian yang bagus dan evaluasi untuk saya.

Tugas dan Latihan Praktikum 2

A. Integrasi Rekam Jejak Penarikan

1. Pada method tarikTunai() ditambahkan sistem untuk mencatat idTrx, jenis, dan nominalnya. Untuk tarikTunai ini jenis transaksinya adalah “Debit” , sedangkan setorTunai jenisnya “Kredit”. Lalu pencatatan ini berhasil jika saldonya tidak 0 dan lebih dari 10000.

```
ic void tarikTunai(double nominal) {
if (saldo <= 10000) {
    System.out.println("Saldo Anda tidak mencukupi untuk menarik tu
} else {
    if (nominal >= saldo) {
        System.out.println("Saldo Anda tidak Cukup!");
    } else {
        saldo -= nominal;
        String idTrx = "TRX-T-" + System.currentTimeMillis();
        transaksi trxBaru = new transaksi(idTrx, "Dedit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Saldo Anda saat ini : Rp" + saldo);
    }
}
```

B. CetakMutasi

1. Pada rekening.java, dibuatkan sebuah method dengan nama cetakMutasi bertipe void. Pada method ini menggunakan lokal variable yang gunanya memberikan penomoran transaksi, jika transaksinya lebih dari 3, transaksi yang di tampilkan hanya 3 transaksi yang terbaru. Lalu perulangan for untuk penomoran dan mengakses index pada objek riwayatTransaksi tersebut untuk menampilkan detail dari transaksi. Jika i nya 0, berarti transaksinya di cetak dari index 0. Tapi kalau i nya 2, berarti pencetakan dimulai dari index 2.\

```java
public void cetakMutasi() {
    System.out.println("-------Riwayat Transaksi-------");
    int start = 0;
    if (riwayatTransaksi.size() > 3)
        start = riwayatTransaksi.size() - 3;

    for (int i = start; i < riwayatTransaksi.size(); i++)
        riwayatTransaksi.get(i).cetakDetail();
}
```

2. Untuk di main, opsinya pada no 6 ditambahkan Cetak Mutasi untuk menampilkan detail dari transaksi setor dan tarik tunai tersebut.
3.

```
Pilih Menu : 2
Masukkan nominal setor : 50000
Setor tunai Rp50000.0berhasil. saldo saat ini Rp60000.0
```

```
Pilih Menu : 3
Masukkan saldo yang di tarik : Rp20000
Saldo Anda saat ini : Rp40000.0
```

4. Lalu untuk detail dari transaksi tersebut ditampilkan menggunakan opsi no 6 yang memanggil method cetakMutasi().

```
Pilih Menu : 6
-------Riwayat Transaksi-------
ID : TRX-S-1789945513922 | Jenis : Kredit | Nominal : Rp50000.0
ID : TRX-T-1789945517786 | Jenis : Dedit | Nominal : Rp20000.0
```

5. Untuk id dengan awalan “TRX-S.....“ dengan jenis “Kredit” itu menandakan transaksi penyetoran tunai dan untuk id dengan awalan “TRX-T ”d engan jenis “Debit” itu transaksi berupa penarikan tunai

Latihan / Tugas Praktikum Modul 3

A. Menambahkan fitur saat membuka rekening.

1. Pada class rekening, kita menambahkan attribut pinbaru dan pinlama.

```
    private String pinbaru;
    private String pinLama;

    ArrayList<transaksi> riwayatTransaksi;
```

2. Pada konstruktor, ditambahkan juga parameter PIN bertipe string. Ini ditambahkan agar saat membuat akun baru, pengguna membuat PIN untuk akun tersebut.

```
public rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
    nomorRekening = nomor;
    namaPemilik = nama;
    saldo = saldoAwal;

    if (pinAwal.length() == 6) {
        this.pinbaru = pinAwal;
        this.pinLama = pinAwal;
    } else {
        System.out.println("Peringatan: PIN harus berisikan 6 digit! menggunakan PIN default 123456");
        this.pinbaru = "123456";
    }

    this.riwayatTransaksi = new ArrayList<>();

    System.out.println("Rekening atas nama : " + namaPemilik + " Berhasil dibuat dengan saldo Rp" + saldo);
}
```

3. Pada class main.java, pilihan 1 ditambahkan variable baru untuk menginputkan PIN baru. Jadi akun akan menyimpan norekening, nama, saldo, dan pin akun tersebut.

```
tch (pilihan) {
 case 1 :
     System.out.print("Masukkan No Rekening :");
     String no = input.nextLine();
     System.out.print("Masukkan Nama Pemilik : ");
     String nama = input.nextLine();
     System.out.print("Masukkan Saldo Awal : ");
     double saldo = input.nextDouble();
     input.nextLine();
     System.out.print("Buat PIN : ");
     String pinAkun = input.nextLine();

     akunSaya.add(new rekening(no,nama,saldo,pinAkun));
     akunAktif = akunSaya.get(noAkun);
     break;
```

4. Untuk tampilan hasilnya seperti berikut :

```
Pilih Menu : 1
Masukkan No Rekening :120105
Masukkan Nama Pemilik : kapallaut
Masukkan Saldo Awal : 15000
Buat PIN : 121212
Rekening atas nama : kapallaut Berhasil dibuat dengan saldo Rp15000.0
```

B. Memperbaharui method tarikTunai dan cetakMutasi

1. Sebelum mengubah atau menambahkan kode pada method tersebut. Sistem perlu membuat method autentikasi agar PIN yang sudah di buat bisa di verifikasi.
2. Jadi tambahkan method otentikasi bertipe boolean agar nilai returnnya menghasilkan true / false dengan parameter PIN(String).

```
public boolean otentikasi(String inputPin) {
    return this.pinbaru.equals(inputPin);
}
```

3. Lalu baru method tarikTunai dan cetakMutasi di perbaharui dengan menambahkan inputanPIN dan pengecekan PIN dengan method otentikasi yang sebelumnya dibuat.

```
case 3:

        if (akunAktif == null) {
            System.out.println("Error : Mohon maaf, Anda belum memiliki nomor rekening!");
        } else {
            System.out.println("Masukkan PIN : ");
            String pinAktif = input.nextLine();
                if (akunAktif.otentikasi(pinAktif)) {
                    System.out.print("Masukkan saldo yang di tarik : Rp");
                    double nominal = input.nextDouble();
                    akunAktif.tarikTunai(nominal);
                    break;
                } else {
                    System.out.println("Akses ditolak! PIN yang anda masukkan salah!");
                }
        }
```

4. Ini untuk tarikTunai(opsi 3).

```
case 6:
    System.out.println("Masukkan PIN : ");
    String pin = input.nextLine();
    if (akunAktif.otentikasi(pin)) {
        akunAktif.cetakMutasi();
    } else {
        System.out.println("Akses ditolak! PIN yang anda masukkan salah!");
    }
    break;
```

5. Ini untuk cetakMutasi(opsi 6)
6. Berikut tampilan dari program tersebut.

```
Pilih Menu : 3
Masukkan PIN :
121212
Masukkan saldo yang di tarik : Rp10000
Saldo Anda saat ini : Rp5000.0
```

7. Ini kondisi saat PIN yang dimasukkan bernilai benar, jadi sistem akan lanjut ke fitur yang diakses pengguna.

```
Pilih Menu : 6
Masukkan PIN :
101010
Akses ditolak! PIN yang anda masukkan salah!
```

8. Ini saat PIN yang dimasukkan salah, sistem akan lansung menolak permintaan pengguna dan mengembalikan ke menu utama.

Link Repository

https://github.com/razif05/PrakPBO_B_2511532028/blob/Modul-3/src/Modul_3

Latihan / Tugas Praktikum Modul 4

A. Menambahkan fitur untuk multi-produk (bisa memilih untuk tabungan atau girobisnis).

1. Pada class main.java , kita menambahkan kode program untuk bisa memilih jenis rekening yang ingin dibuka itu tabungan atau girobisnis. Jadi dibuatkan inputan bertipe int untuk memilih jenis rekening. Untuk menu pilihan tersebut menggunakan switch-case.

```
System.out.print("Pilih tabungan yang ingin di tambahkan: ");
int pilihTab = input.nextInt();
input.nextLine();
switch (pilihTab) {
    case 1:
        System.out.print("Masukkan Suku Bunga(%): ");
        double sukuBunga = input.nextDouble();
        input.nextLine();
        akunSaya.add(new RekeningTabungan(no,nama,saldo,pinAkun,sukuBu
        akunAktif = akunSaya.get(noAkun);
        break;
    case 2:
        System.out.print("Masukkan BatasOverdraft: ");
        double batasOverdraft = input.nextDouble();
        input.nextLine();
        akunSaya.add(new RekeningGiro(no,nama,saldo,pinAkun, batasOver
        akunAktif = akunSaya.get(noAkun);
```

2. Jadi saat user memilih opsi 1. Berarti user memilih rekening tabungan dan harus memasukan sukuBunga. Lalu objek rekeningtabungan dibuat dan ditambahkan ke akunSaya.

```
switch (pilihTab) {
    case 1:
        System.out.print("Masukkan Suku Bunga(%): ");
        double sukuBunga = input.nextDouble();
        input.nextLine();
        akunSaya.add(new RekeningTabungan(no,nama,saldo,pinAkun,sukuBu
        akunAktif = akunSaya.get(noAkun);
        break;
```

3. Jika user memilih opsi 2, user memilih membuka RekeningGiro dan menginputkan batasOverdraft. Saat pilihan diluar 1 atau 2, sistem akan balik ke menu utama dan membatalkan buka rekening.

```
case 2:
    System.out.print("Masukkan BatasOverdraft: ");
    double batasOverdraft = input.nextDouble();
    input.nextLine();
    akunSaya.add(new RekeningGiro(no,nama,saldo,pinAkun, batasOver
    akunAktif = akunSaya.get(noAkun);
    break;
default:
    System.out.println("pilihan tidak valid");
    break;
```

4. Untuk tampilan hasilnya seperti berikut :

```
Pilih Menu : 1
Masukkan No Rekening :12551251
Masukkan Nama Pemilik : nani
Masukkan Saldo Awal : 120000
Buat PIN : 121212
Pilih tabungan yang ingin di tambahkan: 1
Masukkan Suku Bunga(%): 20
Rekening atas nama : nani Berhasil dibuat dengan saldo Rp120000.0
```

```
Pilih Menu : 1
Masukkan No Rekening :w6136116
Masukkan Nama Pemilik : red
Masukkan Saldo Awal : 3162616316
Buat PIN : 232323
Pilih tabungan yang ingin di tambahkan: 2
Masukkan BatasOverdraft: 1000000
Rekening atas nama : red Berhasil dibuat dengan saldo Rp3.162616316E9
```

B. Simulasi Waktu Akhir Bulan (Tugas)

1. Menambahkan menu ke-8 yaitu Simulasi Akhir Bulan (Khusus Tabungan).
2. Lalu didalam pilihn tersebut di periksa menggunakan perintah “if”, akunAktif tersebut diperiksa apakah rekeningnya itu termasuk rekeningtabungan atau bukan.
3. Jika akunnya merupakan rekeningtabungan, akunAktif ini diubah menjadi rekeningTabungan agar bisa mengakses method atau fitur dari si rekeningTabungan tersebut dan disimpan ke dalam TabunganAnda. Lalu method tambahBungaAkhirBulan() dari subclass RekeningTabungan tadi dipanggil melalui TabunganAnda.
4. Jika bernilai false. Sistem memberikan pesan gagal kalau fitur tersebut hanya tersedia untuk rekening tabungan.

```
case 8:
    if (akunAktif instanceof RekeningTabungan) {
        RekeningTabungan TabunganAnda =  (RekeningTabungan) akunAktif;
        TabunganAnda.tambahBungaAkhirBulan();
        break;
    } else {
        System.out.println("Gagal! Fitur bunga akhir bulan hanya berlaku untuk Rekening T
        break;
    }
```

5. Berikut tampilan dari program tersebut.

```
Pilih Menu : 1
Masukkan No Rekening :152151
Masukkan Nama Pemilik : budi
Masukkan Saldo Awal : 12000
Buat PIN : 121212
Pilih tabungan yang ingin di tambahkan: 1
Masukkan Suku Bunga(%): 10
Rekening atas nama : budi Berhasil dibuat dengan saldo Rp12000.0
```

```
Pilih Menu : 8
Bunga 10.0% berhasil ditambahkan: Rp1200.0
```

```
Pilih Menu : 8
Gagal! Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan
```

Ini saat rekeningGiro mengakses fitur khusus rekening tabungan.

Latihan / Tugas Praktikum Modul 5

A. Membuat method transfer pada class rekening.java
   1. Membuat method transfer dengan return type void, lalu parameternya berupa object tujuan (rekening tujuan) dan nominal.
   2. Lalu terdapat logika dari method otentikasi(pinInput) untuk memeriksa pin yang dimasukkan benar ato salah. Setelah itu jika pinnya benar tambahkan logika dengan memeriksa method tarikTunai() agar terhindar dari false yang dihasilkan method tersebut. Lalu jalankan tujuan.setorTunai(nominal) agar saldo rekening tujuan bertambah

```
public void transfer(rekening tujuan, double nominal, String p
    if (!this.otentikasi(pinInput)) {
        System.out.println("Pin yang dimasukkan salah");

    } else {
        if (this.tarikTunai(nominal) ) {
            tujuan.setorTunai(nominal);
        } else {
            System.out.println("Transfer Gagal!");
        }
    }
}
```

   3. Lalu pada main.java sambungkan method tadi ke opsi ke 9 (9. Transfer). Pengguna memasukkan inputan nomor rekening tujuan, lalu di cek apakah nomor rekening tujuan tersebut ada / sesuai dengan nomor rekening yang ada di daftarRekening. Kalo ada, jalankan akunAktif.transfer(tujuan, nominal, pin).

```
case 9:
    System.out.println("Masukkan Rekening tujuan :");
    String rekTujuan = input.nextLine();
    for (int i = 0; i < akunSaya.size(); i++) {
        if (akunSaya.get(i).getNoRek().equals(rekTujuan)) {
            rekening akunTujuan = akunSaya.get(i);
            System.out.println("Masukkan Nominal Transfer :");
            double nominal = input.nextDouble();
            input.nextLine();
            System.out.println("Masukkan PIN :");
            String pinAktif = input.nextLine();
            akunAktif.transfer(akunTujuan, nominal, pinAktif);
        }
    }

    break;
```

   4. Untuk hasilnya seperti berikut :

```
Pilih Menu : 1
Masukkan No Rekening :budi01
Masukkan Nama Pemilik : budi
Masukkan Saldo Awal : 50000
Buat PIN : 121212
Pilih tabungan yang ingin di tambahkan: 2
Masukkan BatasOverdraft: 400000
Rekening atas nama : budi Berhasil dibuat dengan saldo Rp50000.0
```

```
Pilih Menu : 1
Masukkan No Rekening :dani01
Masukkan Nama Pemilik : dani
Masukkan Saldo Awal : 30000
Buat PIN : 212121
Pilih tabungan yang ingin di tambahkan: 1
Masukkan Suku Bunga(%): 20
Rekening atas nama : dani Berhasil dibuat dengan saldo Rp30000.0
```

```
Pilih Menu : 9
Masukkan Rekening tujuan :
dani01
Masukkan Nominal Transfer :
20000
Masukkan PIN :
121212
Tarik tunai Giro Rp20000.0 berhasil. Saldo saat ini Rp30000.0
Setor tunai Rp20000.0 berhasil. saldo saat ini Rp50000.0
```

Link Repository & Video

https://github.com/razif05/PrakPBO_B_2511532028/blob/Modul-5/src/Modul_5

https://drive.google.com/drive/folders/1Et9ByvIE4uiHodaFYBkz7IJAuk_PdE9m?usp=sharing